

token-burner · 文档消耗器

AI 多智能体项目团队系统

产 品 介 绍

（竞赛版 · 纯文字版 · 2026 年 9 月）

一、产品概述

token-burner（文档消耗器）是一套 AI 多智能体项目团队系统：以「主 LLM 架构师 + 开发副 LLM + 测试副 LLM」三角色编队协作，把一句自然语言软件需求，转化为一个可运行、经验证、可审计的完整 Web 项目（代码、测试、接口契约、变更记录与成本看板全程交付）。

与市面单模型编码助手相比，本产品的核心差异化在于三点：其一，逐模块开发循环——需求先拆分为带接口契约的模块，按依赖序逐个「写代码 → 写测试 → 门禁校验 → 执行 → 修复」，错误在产生的模块内就地暴露；其二，七类确定性审计器——以纯程序手段（零 LLM 调用）持续校验生成代码与契约的一致性，由 ARC-Bench 官方容器环境的真实失败取证驱动；其

三，全链韧性工程——单模型超时自动接力编队内 其余模型，单模块失败冻结继续，验收失败优雅退出且现场完整保留。

本产品已完成 ARC-Bench 官方容器环境的六次实弹运行验证，全部过程数据、计费单与缺陷-修复 对照记录公开可查（见第七节）。

二、产品定位与适用场景

2.1 解决的问题

- 黑盒生成：主流工具提示词进去、代码出来，过程不可审计，质量全凭模型状态；
- 集成无人管：文件单看都对，拼在一起 import 断裂、种子数据丢失、接口漂移；
- 成本失控：多模型多轮协作放大 token 消耗，讨论发散、修复死循环、单次输出倾泻使账单不可预期；
- 只写不验：「看起来对」不等于「跑得对」，缺乏真实执行与端到端验证闭环。

2.2 适用场景

- 基准评测与竞技场（ARC-Bench 等「需求 → 可运行应用」类基准）；
- 中小软件团队与外包交付：需要可审计、可预算、可复核的 AI 生成链路；
- 企业内部工具：合规要求全过程留痕、成本上限受控的生成式开发。

三、工作原理

3.1 生产流水线（八段）

评估路由 → 三模型组队 → 方案讨论 → 模块拆分与接口契约 → 逐模块开发循环 → 机械组装 → 两段式验收 → 交付。每一段都有确定性闸门：讨论有轮数与循环检测护栏，拆分产出接口契约，模块开发有执行反馈闭环，验收采用「确定性冒烟 + 旅程闸门」两段制。

3.2 逐模块开发循环

每个模块按依赖拓扑序依次经过五步：写代码（职责与接口契约注入提示词，只写本模块）、写测试（按同一契约生成测试）、接口门禁（代码符号必须与契约一致，防实现后改名）、执行（真实运行本模块测试）、修复/冻结（修复轮耗尽则冻结该模块，管线继续其余模块）。该设计的直接收益：错误在产生的模块内就地暴露；单模块失败冻结继续，部分交付仍可评测，不再全盘归零。

3.3 分层讨论管线

大规模需求树（官方基准 117~143 条原子需求）的全量文本会使讨论阶段超出模型推理时长。本产品将讨论阶段降载为 FOLDER 级摘要（官方 keep 应用实测：17,125 字符 → 4,495 字符，26% 体量），原子级验收细节在拆分与模块开发阶段才回到视野——细节不丢失，只不在讨论阶段挤占推理预算。

四、确定性质量底盘：七类审计器

逐模块门禁能保证「每个模块自身正确」，模块之间的结合部才是生成式系统的主要故障面。本产品的回答不是 更好的提示词，而是七类纯程序审计器：零 LLM 调用、确定性判定，每一类都由一次真实运行失败取证驱动，并在真实交付物上回归验证。

审计器	职责	取证来源
schema 审计	建表 DDL 与 SQL 引用列名一致性 diff（INSERT 清单 / WHERE 谓词 / 限定引用 / SELECT 清单 / 表名漂移，5 类检查）	两次平台实弹运行连续命中
包结构审计	import 名与实际包名映射；空壳包识别	本地 keep 系列
符号级导入审计	from X import sym 静态验证 sym 在 X 内有定义	本地 keep 系列
机械垫片	命名漂移自动生成 re-export 垫片（纯加法，零 LLM 修复）	平台实弹
子模块自动绑定	同名子模块属性缺失自动补绑定	本地 keep 系列
内存哨兵	应用峰值内存超限判不合格（对齐官方 2GB 运行环境）	官方环境裁决
轨迹取证	验收失败自动携带应用侧 traceback（精确到文件行号）注入修复指令	三次实弹定位

配套「修复形态约束」：审计结论注入修复指令时，同步约束修复形态（直接改 SQL 字面量，禁止新增运行时 自适应/兼容层），防止修复模型以过度工程「回应」诊断而非「服从」诊断。

五、模型韧性体系

- 模型链全链保护：单模型调用超时自动接力编队内其余模型，全灭以专用异常类型上抛（备胎调用本身受保护，不再裸奔）；
- 模块级优雅冻结：单模块模型链全灭 → 冻结该模块、管线继续其余模块——部分交付仍进评测， 优于全盘归零；预算超支等闸门类异常不受影响，继续按原语义中止；
- 验收优雅降级：验收失败 → 报告完整保留、干净退出，绝不崩穿宿主进程（官方容器实证）；

- 空内容防线：LLM 空响应当作该模型失败自动接力备胎（修复计划曾因空内容直接死掉）；
- 配置锚定：配置文件随提交包锚定，与运行目录解耦；官方中转站上编制不足三模型时按已知 可用编队自动补全——备胎链的存在不依赖任何外部文件；
- 轨迹取证：冒烟与旅程验收开启异常传播，失败报告自带应用侧完整调用栈。

六、视觉理解通道

需求树内嵌的参考截图（reference/*.png）自动经视觉模型转写为结构化中文描述（布局分区/导航文案/表单 控件/数据元素/配色）注入需求文本。官方环境注入的 VISUAL_MODEL / VISUAL_BASE_URL 视觉端点 优先使用，中转站视觉模型作兜底；带描述缓存（断点不重复计费）、逐级降级与幻觉防线（无效输入绝不上发）。 官方 keep 应用 22 张参考截图实测全识别，描述与真实界面逐项吻合。

七、成本治理与可观测

六层确定性护栏（预算总闸 → 讨论轮数 → 循环检测 → 修复上限 → spec 收敛 → 输出截断）全部由程序执行， 不依赖 LLM 自律；超支即干净中止、现场落盘、支持断点续跑（实战验证）。全程 token 消耗按模型/阶段落盘 logs/cost_report.json，每次实弹运行的平台计费单独立留档。

实测成本基线（Train Ticket 143 需求，官方容器）：单任务 750k~1.2M token， 折合 ¥15.85~29.62/次——失败运行与成功运行的完整计费对照均已留档。

八、实弹验证战绩

战场	结果	说明
SWE-bench Lite	修复前基线 resolved 1/50 (2%)； 环境适配域修复后同域新样本 3/5 (60%)、0 error	环境预检闸门零 token 拦截不可评实例；简化验证口径，同批补丁可交官方 harness 复评
ARC-Bench 平台 (Train Ticket, 143 需求)	官方 Linux 容器内完成讨论→拆分→ 模块开发→交付全流程 (3.5 小时)	分层讨论管线 26% 体量生效；四次运行暴露并修复 12+ 产品缺陷，全部带回归测试
本地 keep 系列 (32 需求)	管线完整交付 8/8 模块；验收拒收点 逐轮前移	六类审计器 + 机械垫片逐轮上线的直接效果

回归测试 1231 项持续全绿（全 stub 设计，无需密钥）；全过程数据、计费单与缺陷-修复对照 见 NIGHT_REPORT.md, git 全程可回溯。

九、路线图

时间	里程碑
9/15-16	keep 系列迭代至全链通过（分分管线 + 七审计器 + 全链保护合体验证）
9/17-18	功能冻结：提交包冻结、模拟环境全流程彩排、提交前检查清单走查
9/21	初赛开题：下载赛题、实弹首单、取得平台真实基线（配额/口径/时限三问已备）
9/22-29	数据驱动迭代：平台反馈 → 修复 → 重交（按配额节奏）
9/30	提交截止：最终版本确认与状态核验

token-burner • 文档消耗器 • AI 多智能体项目团队系统 • 产品介绍（竞赛版） • 2026 年 9 月 • 全文数据口径见 NIGHT_REPORT.md, git 全程可回溯